



Aptos DexRouter

Security Audit Report

Prepared by

OKX Web3 Audit Team

September 2024

Contents

1	Overview	3
1.1	Project Introduction	3
1.2	Audit Summary	3
1.3	Audit Scope	3
2	Audit Summary	4
2.1	Audit Methodology	4
2.2	Audit Process	4
2.3	Risk Classification and Description	5
2.4	Results	5
3	Vulnerabilities	7
3.1	Informational - Check commission_rate before applying commission	7
3.2	Informational - Add empty address validation for referrer_addr	9
3.3	Informational - Non-zero check for slippage protection	12
3.4	Informational - Function visibility can be optimized	15
3.5	Informational - Redundant code can be removed	16
3.6	Informational - Use meaningful strings as resource account seed	18
3.7	Informational - Variable naming can be optimized	19
3.8	Informational - Other suggestions	22
4	Disclaimer	23
5	About OKX Web3 Audit Team	23

1. Overview

1.1 Project Introduction

Aptos DexRouter is a sophisticated decentralized exchange aggregation and routing system built on the Aptos blockchain that enables optimal token swapping across multiple decentralized exchanges (DEXs) and protocols. The system acts as a unified interface for executing complex multi-path swaps, providing users with the best possible rates by intelligently splitting orders across different liquidity sources within the Aptos ecosystem. The router supports both traditional Coin assets and the new Fungible Asset (FA) standard, offering comprehensive commission distribution capabilities for referral programs and revenue sharing mechanisms.

1.2 Audit Summary

Ecosystem	Aptos
Language	Move
Repository	https://github.com/okxlabs/DEX-Router-Aptos-V1
Base Commit	
Final Commit	

1.3 Audit Scope

```
sources/  
├─ aggregator.move  
├─ proxy.move  
└─ router.move
```

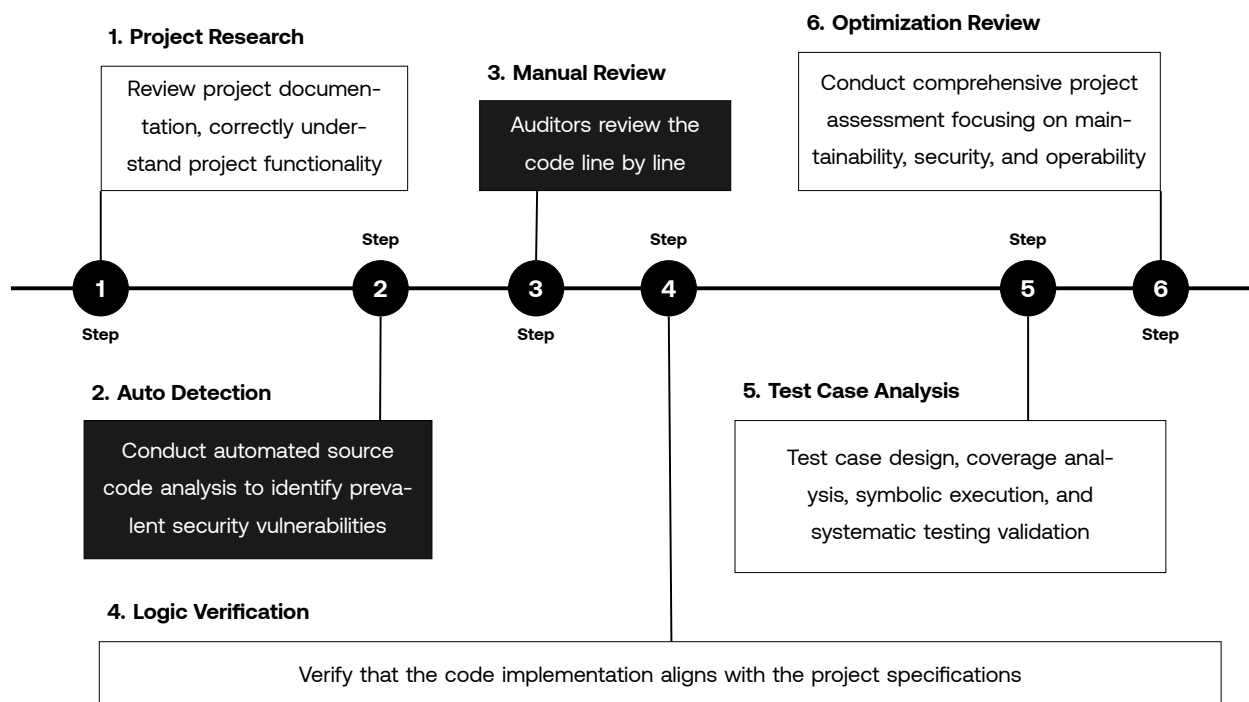
2. Audit Summary

2.1 Audit Methodology

The audit team conducted comprehensive analysis of the contract code through deep understanding of the project’s design purpose, operating principles, and implementation methods. By mapping function call relationships, potential security vulnerabilities were systematically identified, with detailed problem descriptions and corresponding remediation recommendations provided.

2.2 Audit Process

The smart contract security audit follows a 6-phase process: Project Research, Automated Detection, Manual Review, Logic Verification, Test Case Analysis, and Optimization Review. During manual auditing, auditors perform comprehensive code review to identify vulnerabilities and provide detailed solutions. After completing all phases, the lead auditor communicates findings with the project team. Following the team’s responses, we deliver final audit reports to the project team.



2.3 Risk Classification and Description

Risk items are classified into 5 levels: Critical, High, Medium, Low, and Informational. Critical risks require immediate resolution and re-audit before final report delivery; unresolved critical risks result in audit failure.

Risk Level	Risk Description
Critical	Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
High	High risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
Medium	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
Low	Low risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
Informational	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

2.4 Results

The audit results are divided into two parts: one part is the vulnerability summary of the project audit, and the other part is the detailed vulnerability list.

Vulnerability Summary

Critical	High	Medium	Low	Informational	Total
0	0	0	0	8	8

Vulnerability list

No.	Severity	Vulnerability	Category	Status
1	Informational	Check commission_rate before applying commission	Optimization	Fixed
2	Informational	Add empty address validation for refer-rer_addr	Optimization	Fixed
3	Informational	Non-zero check for slippage protection	Optimization	Fixed
4	Informational	Function visibility can be optimized	Optimization	Fixed
5	Informational	Redundant code can be removed	Optimization	Fixed
6	Informational	Use meaningful strings as resource account seed	Optimization	Fixed
7	Informational	Variable naming can be optimized	Optimization	Fixed
8	Informational	Other suggestions	Optimization	Fixed

Status Definitions

- **Open:** The audit team has notified the project team of the vulnerability, but no reasonable remediation has been implemented.
- **Fixed:** The project team has addressed the vulnerability and the fix has been verified by the audit team.
- **Confirmed:** The project team has confirmed awareness of the vulnerability risk but considers it controllable.

3. Vulnerabilities

This section outlines the risk items identified through manual review and auditing tools. Each item includes the specific file path and code location, along with the assigned risk level.

3.1 Informational - Check commission_rate before applying commission

Location	File	Category	Status	Severity
Line 404 & Line 423	aggregator.move	Optimization	Fixed	Informational

Description

If the commission_rate is 0, no commission will be charged. It is recommended to check whether commission needs to be applied before entering the apply_commission function. This can avoid unnecessary function calls and improve code execution efficiency.

Related Code

```

404 // Apply commission for from token if needed
405 if (commission_info.is_from_commission) {
406     apply_commission<X>(&commission_info, *vector::borrow(&asset_objects, 0),
407         ↪ *vector::borrow(&is_FA, 0), x_in);
408 }
409 // ...
410
411 // Apply commission for to token if needed
412 if (commission_info.is_to_commission) {
413     final_output_amount = apply_commission<M>(&commission_info,
414         ↪ *vector::borrow(&asset_objects, 3), *vector::borrow(&is_FA, 3),
415         ↪ final_output_amount);
416 }

```

Recommendation

Add a check for commission_rate != 0 as shown in the following code:

```

1 // Apply commission for from token if needed
2 if (commission_info.is_from_commission && commission_info.commission_rate != 0) {
3     apply_commission<X>(&commission_info, *vector::borrow(&asset_objects, 0),
4         ↳ *vector::borrow(&is_FA, 0), x_in);
5 }
6 // ...
7
8 // Apply commission for to token if needed
9 if (commission_info.is_to_commission && commission_info.commission_rate != 0) {
10     final_output_amount = apply_commission<M>(&commission_info,
11         ↳ *vector::borrow(&asset_objects, 3), *vector::borrow(&is_FA, 3),
12         ↳ final_output_amount);
13 }

```

Project Team Feedback

Team Response	Fixed. Added validation for commission rate.
Re-audit Result	Confirmed fixed

3.2 Informational - Add empty address validation for referrer_addr

Location	File	Category	Status	Severity
Line 214-250	aggregator.move	Optimization	Fixed	Informational

Description

The referrer_addr is the address that receives commission funds. If the commission address is incorrect, the commission funds will be lost. It is recommended to add at least a check to determine whether referrer_addr is an empty address to prevent fund loss.

Related Code

```

179 fun apply_commission<X>{
180     commission_info: &CommissionInfo,
181     asset_object: Object<Metadata>,
182     is_FA: bool,
183     amount: u64
184 }: u64 {
185     // Ensure the commission rate is valid
186     assert!(commission_info.commission_rate <= MAX_COMMISSION_RATE,
187         ↪ E_INVALID_COMMISSION_RATE);
188
189     // Ensure that at most one of is_from_commission and is_to_commission is true
190     assert!(!(commission_info.is_from_commission && commission_info.is_to_commission),
191         ↪ E_INVALID_COMMISSION_CONFIG);
192
193     // If no commission needs to be applied, return the original amount
194     if (!commission_info.is_from_commission && !commission_info.is_to_commission) {
195         return amount
196     };
197
198     // Get PDA
199     let pda = proxy::get_pda_manager();
200
201     // Calculate commission amount
202     let commission_amount = (amount * commission_info.commission_rate) / 10000;
203
204     // Transfer commission amount to referrer
205     let referrer_addr = commission_info.referrer_addr;
206     if (is_FA) {
207         let asset_x = primary_fungible_store::withdraw(&pda, asset_object,
208             ↪ commission_amount);
209         primary_fungible_store::deposit(referrer_addr, asset_x);
210     } else {
211         // Use aptos_account module to avoid the registration check for referrer account
212         aptos_account::transfer_coins<X>(&pda, referrer_addr, commission_amount);
213     };
214
215     // Return the amount after applying commission
216     amount - commission_amount
217 }

```

**Recommendation**

It is recommended to add at least a check to determine whether `referrer_addr` is an empty address:

```
1 assert!(commission_info.referrer_addr != 0x0, E_INVALID_COMMISSION_CONFIG);
```

Project Team Feedback

Team Response	Fixed. Added zero address validation.
Re-audit Result	Confirmed fixed

3.3 Informational - Non-zero check for slippage protection

Location	File	Category	Status	Severity
Line 316	aggregator.move	Optimization	Fixed	Informational

Description

The aggregator.unxswap function receives the min_out parameter, which is used as a minimum value check for the final tokens obtained. However, the function does not limit the value of the min_out parameter. If this value is not set (value is 0), slippage control will not work.

Related Code

```

99 public entry fun unxswap<X, Y, Z, M, E1, E2, E3>(
100     sender: &signer,
101     x_in: u64,
102     min_out: u64,    // @note check greater than 0
103     dex_types: vector<u8>,
104     pool_types: vector<u64>,
105     is_x_to_y: vector<bool>,
106     asset_objects: vector<Object<Metadata>>,
107     is_FA: vector<bool>
108 ) acquires EventEntity {
109     ...
110     public_unxswap<X, Y, Z, M, E1, E2, E3>(dex_types, pool_types, is_x_to_y, asset_objects,
111         ↪ is_FA);
112
113     // check the slippage and transfer output to user
114     // To see if toToken is in FA format
115     if (*vector::borrow(&is_FA, 3)) {
116         let asset_object_3 = *vector::borrow(&asset_objects, 3);
117
118         let asset_y_amount = primary_fungible_store::balance(signer::address_of(&pda),
119             ↪ asset_object_3);
120         assert!(asset_y_amount >= min_out, E_OUTPUT_LESS_THAN_MINIMUM);
121
122         let asset_y = primary_fungible_store::withdraw(&pda, asset_object_3,
123             ↪ asset_y_amount);
124         primary_fungible_store::deposit(signer::address_of(sender), asset_y);
125     } else {
126         let coin_y_amount = coin::balance<M>(signer::address_of(&pda));
127         assert!(coin_y_amount >= min_out, E_OUTPUT_LESS_THAN_MINIMUM);
128
129         let coin_y = coin::withdraw<M>(&pda, coin_y_amount);
130         check_and_deposit(sender, coin_y);
131     }
132 }

```

Recommendation

It is recommended to add a fail-safe design in the function to check that the min_out parameter is not 0, ensuring that slippage control can work properly.



Project Team Feedback

Team Response	Fixed. Resolved legacy issue.
Re-audit Result	Confirmed fixed

3.4 Informational - Function visibility can be optimized

Location	File	Category	Status	Severity
Line 93	aggregator.move	Optimization	Fixed	Informational

Description

Many functions defined within the Aggregator are only called internally within the module. It is recommended to modify these functions that are only used within the module to private visibility to restrict external access. This can not only reduce external contracts' dependence on the module's internal implementation, but also enhance the module's encapsulation and prevent unnecessary external calls.

Related Code

```

154 public fun get_intermediate_output<X, Y, E>( //@audit-info public -> internal allows any
    ↪ module to call this function and obtain PDA signer capability, should be changed to
    ↪ internal
155     dex_type: u8,
156     pool_type: u64,
157     is_x_to_y: bool,
158     asset_object_in: Object<Metadata>,
159     asset_object_out: Object<Metadata>,
160     is_in_FA: bool,
161     is_out_FA: bool
162 ) acquires EventEntity {
163     ...
164 }
```

Recommendation

Optimize function visibility by modifying functions that are only used within the module to private or internal visibility.

Project Team Feedback

Team Response	Fixed. Resolved legacy issue.
Re-audit Result	Confirmed fixed

3.5 Informational - Redundant code can be removed

Location	File	Category	Status	Severity
Line 21	proxy.move	Optimization	Fixed	Informational

Description

The `withdraw_token_left()` function in the proxy module allows transferring remaining token X from the resource account to the admin account. However, since the tokens being transferred are from the resource account, checking whether the resource account has already registered token type X before token transfer is redundant and can be safely removed. Additionally, this function can only extract coin assets and does not support FA asset extraction.

Related Code

```

189 public entry fun withdraw_token_left<X>(sender: &signer) acquires ResourceAccount {
190     let sender_address = signer::address_of(sender);    //@audit: does not support FA asset
191     ↪ extraction
192     assert!(sender_address == @metaxtrade, E_NOT_ADMIN);
193     let resourceAccount = borrow_global<ResourceAccount>(@metaxtrade);
194     let accountSinger =
195     ↪ account::create_signer_with_capability(&(resourceAccount.signer_cap));
196
197     // register token for admin and pda if one of them has not registered
198     if (!coin::is_account_registered<X>(sender_address)) {
199         coin::register<X>(sender)
200     };
201     if (!coin::is_account_registered<X>(signer::address_of(&accountSinger))) {
202         ↪ //@audit-info : reduandant
203         coin::register<X>(&accountSinger)
204     };
205
206     coin::transfer<X>(&accountSinger, sender_address ,
207     ↪ coin::balance<X>(signer::address_of(&accountSinger)));
208 }

```

Recommendation

Remove the check for whether the resource account has already registered token type X before token transfer. Additionally, add support for FA asset extraction.



Project Team Feedback

Team Response	Fixed redundant code and added support for FA asset extraction.
Re-audit Result	Confirmed fixed

3.6 Informational - Use meaningful strings as resource account seed

Location	File	Category	Status	Severity
Line 39	router.move	Optimization	Fixed	Informational

Description

When auditing the `metaxtrade::router::init()` function, it was found that `111` was used as the seed when creating the resource account, which is not a good programming practice. It is recommended to use meaningful strings as seeds, such as strings like `b"okx.dex.escrow.account"`. This can not only increase the readability and maintainability of the code, but also clearly express the purpose of the resource account, avoiding confusion caused by using meaningless constant values.

Related Code

```
228 public entry fun init(admin: &signer) {
229     assert!(signer::address_of(admin) == @metaxtrade, E_NOT_ADMIN);
230
231     // init router
232     let seed = vector::empty<u8>();
233     vector::push_back<u8>(&mut seed, 111); // @audit-info recommend not using 111 as seed
234     proxy::init(admin, seed);
235 }
```

Recommendation

Use meaningful strings as the seed for the resource account, such as `b"okx.dex.escrow.account"`.

Project Team Feedback

Team Response	Fixed. Resolved legacy issue.
Re-audit Result	Confirmed fixed

3.7 Informational - Variable naming can be optimized

Location	File	Category	Status	Severity
Line 225 & Line 249	aggregator.move	Optimization	Fixed	Informational

Description

The naming order of the is_FA flag variables in this update is different from the asset_objects variable naming method in the previous version. It is recommended to unify the naming format according to the index number in the array.

Related Code

```

261 public fun public_unxswap<X, Y, Z, M, E1, E2, E3>(
262     dex_types: vector<u8>,
263     pool_types: vector<u64>,
264     is_x_to_y: vector<bool>,
265     asset_objects: vector<Object<Metadata>>,
266     is_FA: vector<bool>
267 ) acquires EventEntity {
268     assert!(vector::length(&dex_types) == vector::length(&pool_types), E_LENGTH_NOT_EQUAL);
269     assert!(vector::length(&pool_types) == vector::length(&is_x_to_y), E_LENGTH_NOT_EQUAL);
270
271     assert!(vector::length(&asset_objects) == FA_INFO_LENGTH, E_INVALID_FA_INFO_LENGTH);
272     assert!(vector::length(&is_FA) == FA_INFO_LENGTH, E_INVALID_FA_INFO_LENGTH);    //
    ↪ @note 4
273     assert!(vector::length(&dex_types) <= 3, E_OUT_HIP);
274     // Add necessary params extraction from vector
275     if (vector::length(&dex_types) == 1) {
276         let dex_type_0 = *vector::borrow(&dex_types, 0);
277         let pool_type_0 = *vector::borrow(&pool_types, 0);
278         let is_x_to_y_0 = *vector::borrow(&is_x_to_y, 0);
279         let asset_object_0 = *vector::borrow(&asset_objects, 0);
280         let asset_object_3 = *vector::borrow(&asset_objects, 3);
281         let is_FA_0 = *vector::borrow(&is_FA, 0);
282         let is_FA_1 = *vector::borrow(&is_FA, 3);    // @audit recommend unifying with
    ↪ asset_object_3 numbering, change to is_FA_3
283         one_step_internal<X, M, E1>(
284             dex_type_0,
285             pool_type_0,
286             is_x_to_y_0,
287             asset_object_0,
288             asset_object_3,
289             is_FA_0,
290             is_FA_1
291         );
292     }
293     // ...
294 }

```

Recommendation

It is recommended to unify the variable name corresponding to `borrow(&is_FA, 3)` with `asset_object_3` numbering, changing it to `is_FA_3`.

**Project Team Feedback**

Team Response	Fixed.
Re-audit Result	Confirmed fixed

3.8 Informational - Other suggestions

Location	File	Category	Status	Severity
Multiple locations	Multiple Contracts	Optimization	Fixed	Informational

Description

The following lists other optimization points in the DexRouter contract code to improve code style and readability:

1. Add is_FA length validation at function entry to ensure its length is 4
2. Function input parameter is_x_to_y is not used and can be safely removed (but the project team chose to keep this parameter for future potential compatibility support)
3. Recommend consolidating PDA-related operations in the proxy module

Related Code

```

1 // Various optimization points throughout the codebase
2 // 1. is_FA length validation at function entry
3 // 2. Unused is_x_to_y parameter (kept for compatibility)
4 // 3. PDA operations consolidation in proxy module

```

Recommendation

Modify the optimizable parts of the code to improve code quality and maintainability.

Project Team Feedback

Team Response	1. Original is_FA length check was inside the function, now all checks have been moved to function entry; 2. DexRouter parameters need to consider support for all Dex protocol parameters, chose to keep this parameter for future potential compatibility support; 3. Fixed legacy issue.
Re-audit Result	Confirmed fixed

4. Disclaimer

This audit report only covers the specific audit types stated herein. We assume no responsibility for unknown security vulnerabilities outside this scope.

We rely on audit reports issued before existing attacks or vulnerabilities are published. For future or new vulnerabilities, we cannot guarantee project security impact and assume no responsibility.

Our security audit analysis should be based on documents provided by the project before report release (including contract code). These materials should not contain false information, tampering, deletion, or concealment. If provided materials are false, inaccurate, missing, tampered, deleted, concealed, or modified after report release, we assume no responsibility for resulting losses and adverse effects.

The project team should understand our audit report is based on provided materials and current technical capabilities. Due to institutional technical limitations, our report may not detect all risks. We encourage continued testing and auditing by the development team and stakeholders.

The project team must ensure compliance with applicable laws and regulations.

The audit report is for reference only. Its content, acquisition methods, usage, and related services cannot serve as basis for investment, taxation, legal, regulatory, or construction decisions. Without our prior written consent, the project team may not reference, cite, display, or distribute report content to third parties. Any resulting losses shall be borne by the project team.

This report does not cover contract compiler bugs or scope beyond programming languages. Smart contract risks from underlying vulnerabilities should be borne by the project team.

Force majeure includes unforeseeable, unavoidable events like wars, natural disasters, strikes, epidemics, and legal/regulatory changes preventing contract performance. When occurring, neither party breaches contract obligations. For unaffected economic responsibilities, the project team should pay for completed work.

5. About OKX Web3 Audit Team

OKX Web3 Audit Team specializes in blockchain security with expertise in smart contract auditing, token security assessment, and Web3 security tool development. We provide comprehensive security solutions for OKX's internal Web3 projects, conduct pre-listing token audits, and develop security tools to protect OKX Web3 wallet users. Our team combines automated analysis with manual review to deliver thorough security assessments and maintain the highest security standards in the Web3 ecosystem.